DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Jungle Jump Pt. 1

CMSC 197 GDD Activity 5

Prepared by: Rene Andre B. Jocsing

Published: March 10, 2026

Session Objectives

- Configure project for pixel art rendering and physics layers
- Understand CharacterBody2D vs RigidBody2D architectural trade-offs
- Build player scene using AnimationPlayer for complex animations
- Implement Camera2D with proper zoom and limits
- Introduce finite state machine skeleton for player behavior

---

Project Setup

Create New Project

1. Open Godot, click New Project
2. Name: `JungleJump`
3. Create project folder
4. Click Create & Edit

Download Assets

Download the project assets from this Drive and extract to your project folder. Verify `res://assets/` contains the player sprite sheet and other required files.

Configure Rendering

Project, Project Settings, Advanced toggle (top right):

Rendering/Textures:

```
Canvas Textures/Default Texture Filter: Nearest
```

Design Decision: Nearest Filtering

Pixel art requires `Nearest` (point) filtering instead of `Linear` (bilinear) to preserve hard edges. Linear filtering creates blur when scaling pixel art, destroying the aesthetic.

Setting this globally avoids configuring each sprite individually.

Display/Window:

```
Stretch/Mode: canvas_items
Stretch/Aspect: expand
```

Rationale: `canvas_items` maintains pixel-perfect scaling. `expand` allows resizing while preserving image quality. Test this later by resizing the game window during play.

Collision Layers

Project Settings, Layer Names, 2D Physics:

Name the first four layers:

1. `player`
2. `environment`
3. `enemies`
4. `items`

Architecture Note: Layer Naming

Godot's collision system uses bit masks. Naming layers makes Inspector checkboxes readable and prevents layer confusion in large projects.

Without names, you'd be checking "Layer 1" vs "Layer 2" with no semantic meaning.

Input Map

Project Settings, Input Map:

Create these actions with specified keys:

Action	Keys
`right`	D, →
`left`	A, ←
`jump`	Space
`up`	W, ↑
`down`	S, ↓

Critical: Use exact names. These will be referenced in code.

Kinematic Bodies vs Rigid Bodies

Design Decision: CharacterBody2D

Why not RigidBody2D for platformers?

RigidBody2D provides realistic physics simulation:

- Affected by global gravity automatically
- Collision response handled by physics engine
- Friction, bounce, mass all simulated

Problem: Realism ≠ responsive platformer feel.

Players expect:

- Instant direction changes (no momentum)
- Precise jump heights
- Predictable collision response

Solution: CharacterBody2D (kinematic physics)

- No automatic physics simulation
- You implement gravity, friction in code
- `move_and_slide()` provides collision detection without simulation
- Precise control over movement feel

Key Methods:

- `move_and_collide()`: Returns collision info, you handle response
- `move_and_slide()`: Auto-slides along surfaces, detects floor/ceiling

For platformers, `move_and_slide()` is standard. It handles slope sliding and floor detection automatically.

Common Pitfalls

Never directly set `position` on CharacterBody2D!

This bypasses collision detection. Always use movement methods. We'll see proper position manipulation via `_integrate_forces()` later.

Player Scene Setup

Node Hierarchy

```
Player (CharacterBody2D)
|-- Sprite2D
|-- CollisionShape2D
|-- AnimationPlayer
`-- Camera2D
```

Create Root Node

1. New scene, Add Node (Ctrl+A)
2. Select `CharacterBody2D`
3. Rename to `Player`
4. Save scene: `player.tscn` in new `player/` folder
5. **Group children** (critical step!)

Verify Motion Mode and Up Direction in Inspector:

- Motion Mode: `Grounded`
- Up Direction: `(0, -1)`

Grounded mode treats one direction as floor (opposite as ceiling, others as walls). Up Direction determines which.

Collision Layer Configuration

Player node, Inspector:

- Collision/Layer: `player` (layer 1)
- Collision/Mask: `environment`, `enemies`, `items`

Design Pattern: Layer vs Mask

Layer: What physics layers this body occupies

Mask: Which layers this body can interact with

Player is **on** the player layer so enemies/items can detect it. Player **detects** environment/enemies/items via mask.

This decoupling allows fine-grained collision control (e.g., enemy bullets ignore other enemies).

Animations with AnimationPlayer

Why AnimationPlayer vs AnimatedSprite2D?

AnimatedSprite2D: Frame-based texture swapping only

AnimationPlayer: Animates **any property** of **any node**:

- Position, rotation, scale
- Modulate (color/transparency)
- Custom properties
- Multiple nodes simultaneously

For complex characters with property animations beyond texture changes, AnimationPlayer is superior.

Sprite Setup

1. Add `Sprite2D` child to Player
2. Texture: `res://assets/player_sheet.png`
3. Animation section, HFrames: 19
4. Frame: 7 (preview standing pose)

Position: `(0, -16)`

Rationale: Moves sprite upward so feet align with Player node's origin. This makes ground collision logic cleaner—node position represents feet location.

Create Animations

Add `AnimationPlayer` child to Player.

Animation Panel Anatomy:

- Animation dropdown: Select/create animations
- Length: Total animation duration
- Loop toggle: Repeat animation
- Scrubber: Timeline position
- Track list: Properties being animated
- Keyframe icons in Inspector: Add keyframes (hint: select Sprite2D node and look at the inspector, locate the key icons)

Create `idle` animation:

1. AnimationPlayer, Animation, New: `idle`
2. Length: 0.4
3. Enable Loop
4. Select Sprite2D, set Frame to 7
5. Click key icon next to Frame property
6. Create track, leave defaults
7. Move scrubber to 0.3, set Frame to 10, click key

8. Set track Update Mode to `Continuous`
9. If you can't see Update Mode icon, add the keyframes track first.

Repeat for remaining animations:

Name	Length	Frames	Loop
idle	0.4	7 → 10	On
run	0.5	13 → 18	On
hurt	0.2	5 → 6	On
jump_up	0.1	11	Off
jump_down	0.1	12	Off

Collision Shape

1. Add `CollisionShape2D` child
2. Shape: New RectangleShape2D
3. Size to cover sprite, slightly narrower than sprite width
4. Position: `(0, -10)`

Design Pattern: Generous Hitboxes

Collision shapes **smaller than sprites** feel better in platformers. Players shouldn't die from pixels barely touching obstacles.

For player: Slightly smaller hitbox = more forgiving

For enemies: Slightly smaller hitbox = easier to avoid

For collectibles: Slightly larger hitbox = easier to grab

This reduces frustration and improves game feel.

Camera2D Setup

Add `Camera2D` child to Player.

Configuration:

- Enabled: On
- Zoom: `(2.5, 2.5)`

Visual: Pink/purple rectangle in the 2D Scene View shows camera viewport.

Design Decision: Camera Parenting

Camera is **child of Player**, not Main scene. This creates automatic follow behavior—camera moves with player by transform inheritance.

Alternative: Camera in Main scene with manual position updates via code. More flexible but more complex.

For simple follow cameras, parenting is cleaner. Adhere to KISS.

Zoom values:

- < 1: Zoom out (see more world)
- 1: Zoom in (see less world, larger sprites)
- `(2.5, 2.5)`: Pixel art remains sharp due to Nearest filtering

Player State Machine

The Boolean Flag Problem

Naive approach:

```
var is_jumping: bool = false
var is_running: bool = false
var is_crouching: bool = false
```

Problem: What if `is_jumping` and `is_crouching` are both true? Invalid state. Code becomes defensive spaghetti checking flag combinations.

Design Pattern: Finite State Machine

FSM Principle: Entity exists in exactly **one state** at any time.

Benefits:

- Impossible to be in conflicting states
- Clear state transition rules
- Easier debugging (print current state)
- Scales to complex behavior

Simple Implementation: Enum for states, match statement for transitions.

Pro Implementation: Custom classes that implement a State interface. A StateMachine that orchestrates through these classes.

For now, we stick with the simple implementation.

Player State Diagram

States:

- `IDLE`: Inactive
- `RUN`: Running
- `JUMP`: Jumping
- `HURT`: Hurt
- `DEAD`: Dying

Transitions: Will be implemented in next session. For now, create skeleton.

Initial Script

Attach script to Player node, uncheck Template.

```
class_name Player extends CharacterBody2D

# Movement parameters
# Export allows setting of these parameters
# in the editor per node instance.
@export var gravity: int = 750
@export var run_speed: int = 150
@export var jump_speed: int = -300


# Naive state machine
```

```gdscript
enum {IDLE, RUN, JUMP, HURT, DEAD}
var state: int = IDLE


# Upon initialization of node, change state to ALIVE
func _ready() -> void:
    change_state(IDLE)

## Call this to change the player's state.
func change_state(new_state: int) -> void:
    state = new_state
    match state:
        IDLE:
            $AnimationPlayer.play("idle")
        RUN:
            $AnimationPlayer.play("run")
        HURT:
            $AnimationPlayer.play("hurt")
        JUMP:
            $AnimationPlayer.play("jump_up")
        DEAD:
            hide()
```

Architecture Note: Enum States

`enum {IDLE, RUN, ...}` is shorthand for:

const IDLE: int = 0

const RUN: int = 1

Using enum improves readability. `state == IDLE` is clearer than `state == 0`.

Testing Setup

Player scene is not yet playable (no movement code). Create test environment:

1. New scene, root: `Node` named `Main`
2. Add Player instance (Right Click + Instantiate Child Scene)
3. Menu, Debug, Visible Collision Shapes (On)

Run scene (F6): Player should be visible standing with its collider. No movement yet, aside from the animation.

Session Summary

What We Built

- Project configured for pixel art and organized collision layers
- Player scene with AnimationPlayer-driven multi-frame animations
- Camera system with proper zoom
- FSM skeleton for state management

Key Architectural Decisions

1. **CharacterBody2D over RigidBody2D:** Precise control for platformer feel

2. **AnimationPlayer over AnimatedSprite2D:** Property animation flexibility
3. **FSM over boolean flags:** Eliminates invalid state combinations
4. **Camera parenting:** Automatic follow via transform inheritance

Next Session Preview

- Implement gravitational pull in `_physics_process` manually
- Implement player movement with `move_and_slide()`
- Complete state machine with transitions
- Add jump mechanics and gravity
- Create player health system skeleton